

The Transformer: Self-Attention and Architecture

Scaler School of Technology | Deep Learning I | Week 7 — Session 1/2 | Post-Lecture Review & Practice



Key Takeaways

1 Self-Attention eliminates sequential processing

Every token attends to every other token in parallel, computing relevance scores via $\text{softmax}(QK^\top / \sqrt{d_k})V$. This removes the RNN bottleneck entirely and enables $O(1)$ path length between any two positions.

2 Multi-Head Attention captures diverse relationships

Running h parallel attention heads (each with separate W^Q, W^K, W^V projections) lets the model attend to different representation subspaces — syntax in one head, coreference in another, semantics in a third.

3 Positional Encoding restores sequence order

Since attention is permutation-invariant, sinusoidal positional encodings are added to input embeddings. The sin/cos functions at different frequencies give each position a unique signature and allow the model to generalize to unseen sequence lengths.

Test Your Understanding

Try answering these without looking at the slides. Answers are on page 3.

- Q1.** Why do we divide the dot product $QK^\top$ by $\sqrt{d_k}$ before applying softmax?
- Q2.** If a Transformer has 8 attention heads with $d_{\text{model}} = 512$, what is d_k per head?
- Q3.** What would happen if we removed positional encoding entirely from a Transformer trained on language?
- Q4.** Explain why the decoder uses *masked* self-attention but the encoder does not.
- Q5.** Compare the computational complexity of self-attention vs. a recurrent layer for a sequence of length n with model dimension d .

Work through these in a Colab notebook. No starter code — figure out the approach from what you learned in the lecture.

Exercise 1: Implement Scaled Dot-Product Attention

Write a function that takes Q , K , V tensors (shape: $\text{batch} \times \text{seq_len} \times d_k$) and returns the attention output and attention weights matrix. Use PyTorch. Verify on a 4-token sentence that your attention weights sum to 1 along the key dimension. Visualize the weights as a heatmap.

Exercise 2: Build Multi-Head Attention from Scratch

Implement a `MultiHeadAttention` class with $h = 4$ heads and $d_{\text{model}} = 64$. Include the linear projections for Q , K , V and the output projection. Feed in a random ($\text{batch}=2$, $\text{seq}=8$, $\text{dim}=64$) tensor and verify the output shape matches the input shape.

Exercise 3: Visualize Positional Encodings

Generate sinusoidal positional encodings for positions 0–99 with $d_{\text{model}} = 128$. Plot a heatmap of the encoding matrix. Then compute and plot the cosine similarity between position 0 and all other positions — verify that nearby positions have higher similarity.

Challenge Problem

Build a Minimal Transformer Encoder that classifies IMDB reviews as positive/negative:

1. Implement positional encoding + 2-layer Transformer encoder (2 heads, $d_{\text{model}} = 128$)
2. Use mean-pooling over sequence positions for the classification token
3. Train on a subset of IMDB (5000 samples) for 5 epochs
4. Report accuracy and plot training loss

Bonus: Compare against a single-layer LSTM baseline with the same hidden dimension. Which converges faster? Which achieves higher accuracy?

- **The Original Paper:** Vaswani et al. (2017), “*Attention Is All You Need*” — Focus on Sections 3.1–3.3.
- **Illustrated Guide:** Jay Alammar, “*The Illustrated Transformer*” — Best visual walkthrough with step-by-step diagrams.
- **Formal Analysis:** Phuong & Hutter (2022), “*Formal Algorithms for Transformers*” — Rigorous pseudocode for implementation.
- **Positional Encoding:** Kazemnejad (2019), “*Transformer Architecture: The Positional Encoding*” — Deep dive into sin/cos properties.

Online Resources & Tools

- **BerTViz** — Attention head visualization
- **Annotated Transformer** — Harvard NLP PyTorch impl.
- **Lilian Weng’s Blog** — All attention variants
- **3Blue1Brown** — Visual attention explainer
- **PyTorch nn.Transformer** — Official docs
- **Hugging Face** — Production implementations

Fill the Gaps — Prerequisite Refreshers

Felt lost on a specific concept? These resources will help you catch up.

- **Confused about softmax?** 3Blue1Brown neural networks Ep. 3 — converts logits to probabilities for attention weights.
- **Matrix multiplication shaky?** Khan Academy linear algebra — every attention computation is a batched matmul.
- **Forgot Bahdanau attention?** Revisit Week 6 Session 2 — Transformers generalize it to every-position-to-every-position.
- **Unsure about residual connections?** Review Week 4 ResNet lecture — Transformers use “Add & Norm” for the same gradient-flow reasons.

Answers — Test Your Understanding

- A1.** Large d_k causes dot products to grow, pushing softmax into saturation (near-zero gradients). Dividing by $\sqrt{d_k}$ keeps variance ≈ 1 , ensuring well-behaved gradients.
- A2.** $d_k = d_{\text{model}}/h = 512/8 = 64$. Each head operates in a 64-dim subspace; concatenation returns to 512.
- A3.** The model would treat “dog bites man” and “man bites dog” identically — word order is lost without positional information.
- A4.** The decoder must not see future tokens (autoregressive). Masking sets future positions to $-\infty$ before softmax. The encoder sees the full input, so no mask is needed.
- A5.** Self-attention: $O(n^2d)$; Recurrent: $O(nd^2)$. Attention is cheaper when $n < d$, costlier for very long sequences.

Hands-on notebook: `transformer_self_attention_notebook.ipynb`

Instructor: Priyansh Saxena